PID Control: Tuning and implementation

R0005E - Measurements and Feedback Control

Wolfgang Birk

Department of Computer Science, Electrical and Space Engineering

LULEÅ UNIVERSITY OF TECHNOLOGY

Tuning

We have seen that the PID controller has the three parameters k_P, k_I and k_D in the following so called parallel structure:

$$D(s) = k_P + \frac{k_I}{s} + k_D s$$

Tuning implies that the controller parameters are adjusted to yield a certain behavior of the closed loop system. Tuning is not a "one-shot" method and could result in an iterative procedure.

Note:

- We want to avoid an iterative procedure as much as possible, as the outcome is quite unpredictable and time consuming!
- Tuning methods are rule of thumb methods and might not produce satisfactory results in all cases.

Lab C

We will now have a look at three tuning methods:

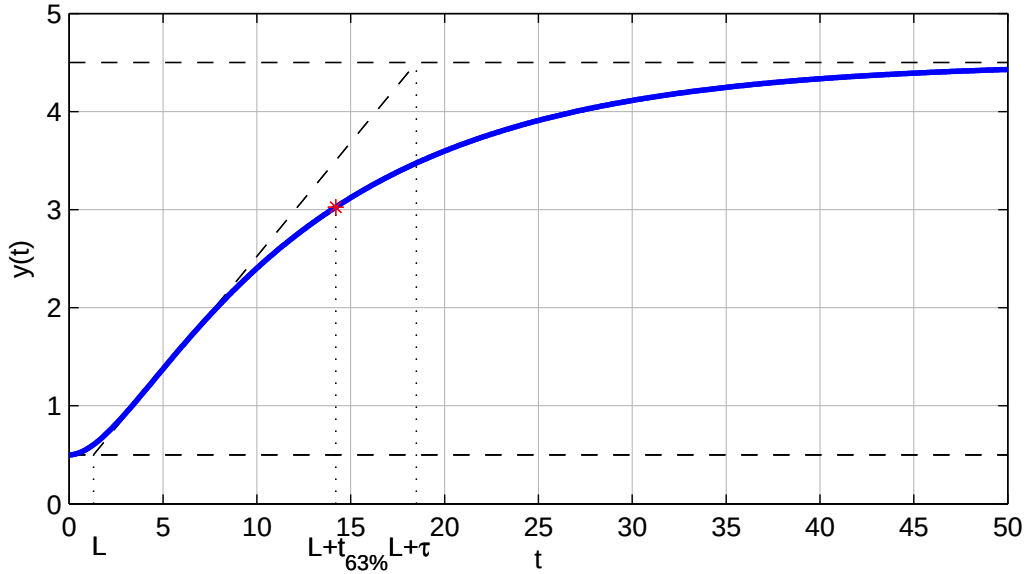
- Ziegel-Nichols quarter decay ratio method.
- Ziegler-Nichols ultimate sensitivity method.
- λ-method.

For those methods it is assumed that the controller is given in the so called ideal structure:

$$D(s) = k_P \left\{ 1 + \frac{1}{T_I s} + T_D s \right\}, \text{ with } k_I = \frac{k_P}{T_I}, \quad k_D = k_P T_D$$

Process reaction curve

From real life experiments it was seen that many processes exhibit a very similar behavior, which can be depicted by the following example:



This curve is called a process reaction curve and can be approximated by the transfer function:

$$G_0(s) = \frac{A}{\tau s + 1} e^{-Ls}$$

If we now know the parameters A, τ and L, then a controller could be designed for the approximation.

Real process: $G(s) = \frac{4}{26s^2+14s+1}$, thus the time constants are $T_1 \approx 2.2$ and $T_2 \approx 11.8$.

Ziegler-Nichols: Quarter decay ratio method

Procedure:

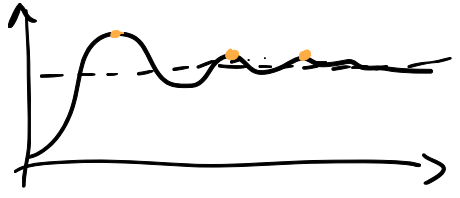
1. Approximate the behavior of the process with the process reaction curve
2. Derive the parameters A, τ and L from the step response
3. Compute the parameters for a P, PI or PID using the reaction rate $R = A/\tau$ as follows:

Controller type	Controller parameters
P	$k_P = \dfrac{1}{RL}$
PI	$k_P = \dfrac{0.9}{RL}, \qquad T_I = \dfrac{L}{0.3}$
PID	$k_P = \dfrac{1.2}{RL}, \qquad T_I = 2L, \qquad T_D = 0.5L$

Resulting performance:

- The closed loop system should have a damping coefficient of approximately $\zeta = 0.21$. This means the oscillations decay at a rate of 0.25.
- The closed loop system is often experienced as being too oscillative.
- Re-tuning might be necessary. In that case, reduce k_P and increase both T_I and T_D.

Mycket bra skrivbok att lösa problemet.



Ziegler-Nichols: Quarter decay ratio method (2)

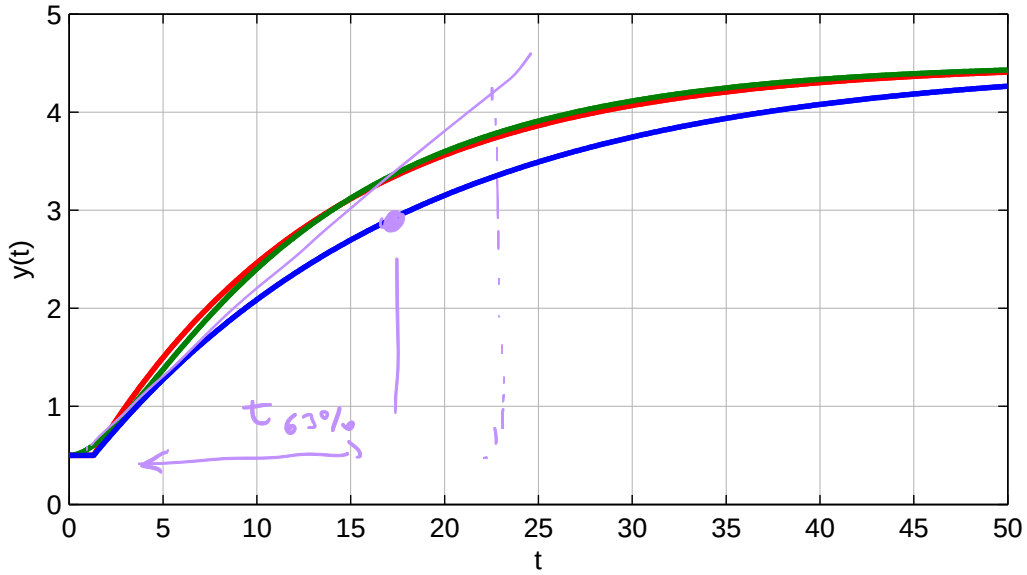
Approximation of the example:

The difficulty is to find a good value for the τ, which affects the reaction rate R.

Using the 63% level to identify τ yields a very different value than the one identified using the slope of the initial reaction:

$$\tau_{slope} = 17.2s, \qquad \tau_{63\%} = 12.9s$$

The remaining values can be determined to be $A = 4$ and $L = 1.3$.



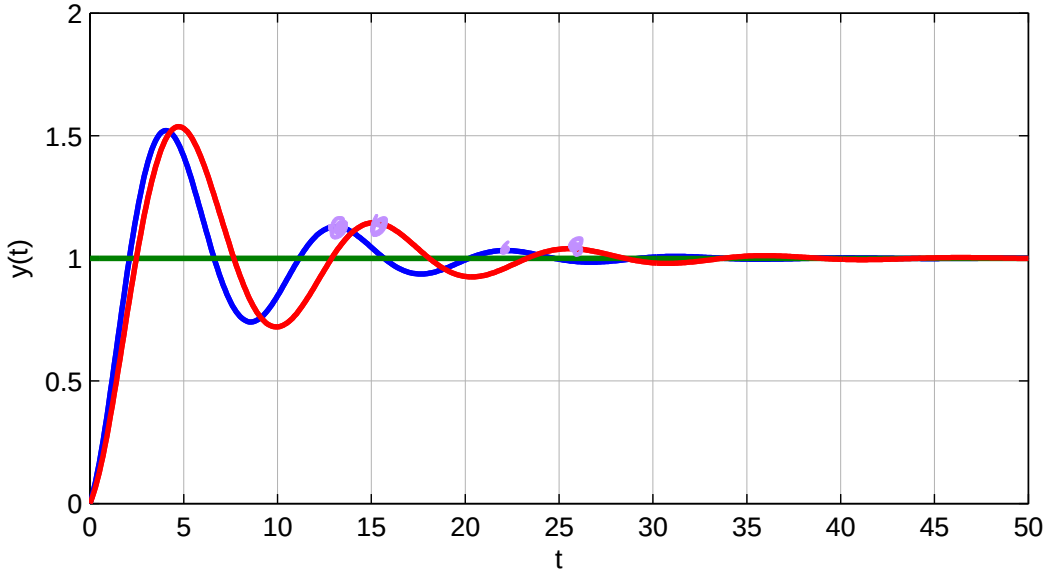
In the figure, the real process (green), approximation with $\tau_{63\%}$ (red) and approximation with τ_{slope} (blue).

Ziegler-Nichols: Quarter decay ratio method (3)

Using the table lookup, the PID controllers for both cases can be found:

$$D_{slope}(s) = \frac{1.5s^2 + 3.9s + 1.5}{s}, \qquad D_{63\%}(s) = \frac{1.1s^2 + 2.9s + 1.1}{s}$$

A simulation of the closed loop system yields the following two responses:



Here, reference value (green), output using $D_{slope}(s)$ (blue) and output using $D_{63\%}(s)$ (red).

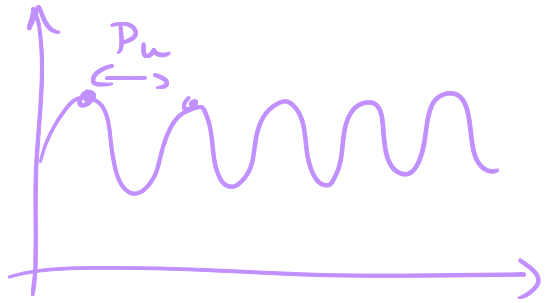
Conclusion: Even though both time constants are rather different, the closed loop system respond similarly and with a correct decay rate.

Ziegler-Nichols: Ultimate sensitivity method

In the quarter decay ratio method, there was not information from closed loop experiments used. This could lead to badly tuned controllers. As a alternative the ultimate sensitivity method was proposed. In this case, the process reaction curve is not used.

1. Use a P controller for the process and increase its gain until the closed loop system oscillates undamped.
2. Record the ultimate gain K_u of the controller and the ultimate period P_u at which the closed loop system oscillates.
3. Compute the parameters for a P, PI or PID using K_u and P_u as follows:

Controller type	**Controller parameters**
P	$k_P = 0.5K_u$
PI	$k_P = 0.45K_u, \qquad T_I = \frac{P_u}{1.2}$
PID	$k_P = 0.6K_u, \qquad T_I = 0.5P_u, \qquad T_D = 0.125P_u$



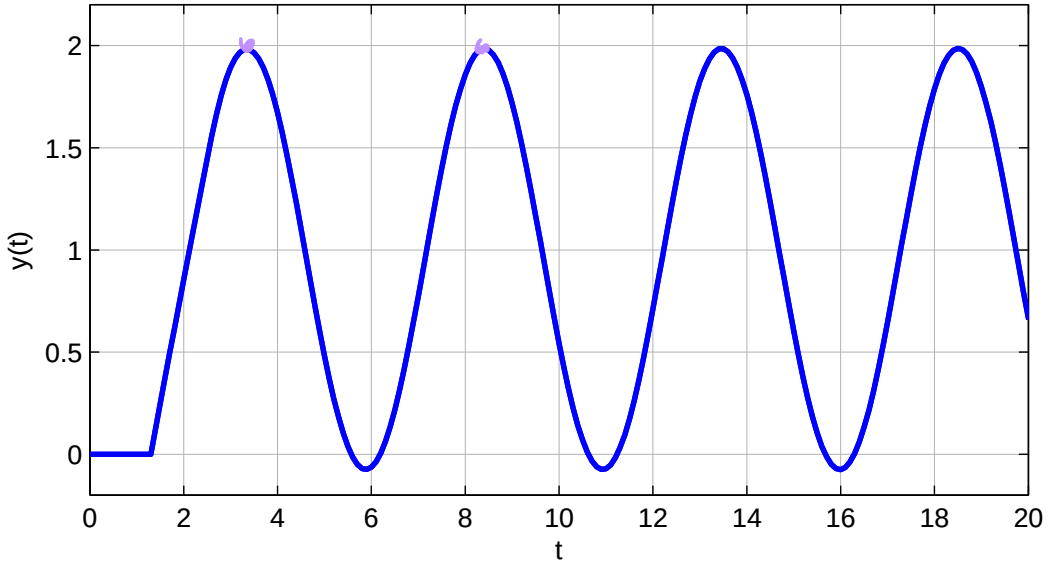
Ziegler-Nichols: Ultimate sensitivity method (2)

Resulting performance:

- This tuning method should perform better than the quarter decay ratio method.
- If the loop might oscillate too much then re-tuning need to be performed in a similar way as before.
- It can be difficult/impossible to perform these types of experiments in real-life.

Analysis of the example:

Using a simulation experiment, the control loop is brought to the stability border:

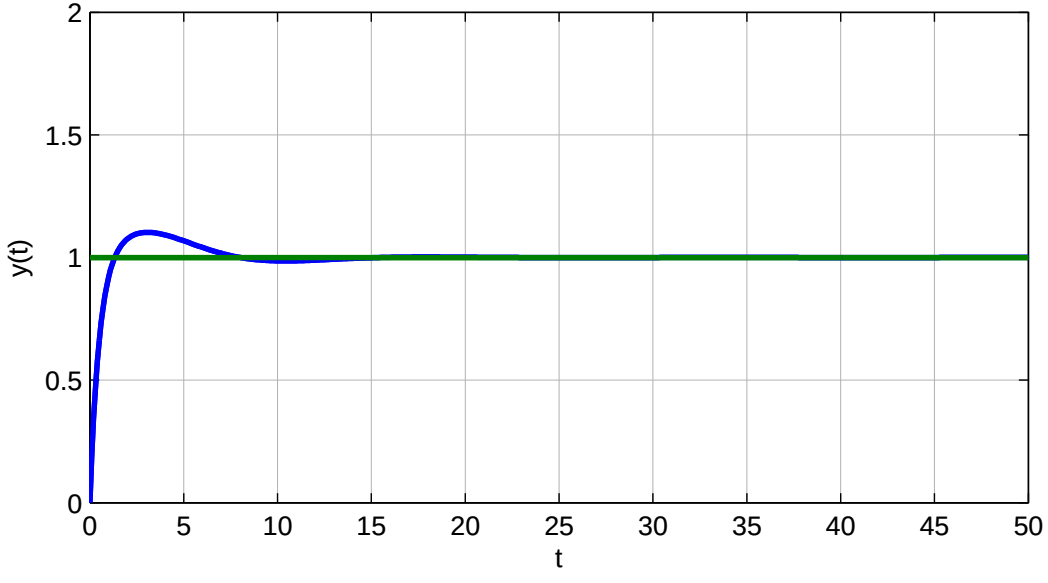


Ziegler-Nichols: Ultimate sensitivity method (3)

From the simulation settings and the resulting plots, the ultimate gain is $K_u = 5.4$ and the ultimate period is $P_u = 5.1s$. The resulting PID controller is found as

$$D(s) = \frac{13.4s^2 + 8.6s + 3.4}{s}$$

The resulting closed loop system yields the following step response:



Note: This controller performs much better, but it also has a much higher gain, than the controllers $D_{63\%}(s)$ and $D_{slope}(s)$. This could lead to stability issues when the process changes.

λ-Method

The Ziegler-Nichols method was proposed in 1942 and 1943. A more modern tuning method is the λ-method. It is originally derived form the so call internal model control concept (IMC) as a special case.

The method is intended to compute parameters for PI controllers alone. The method is now the industry standard in the pulp and paper industry.

Procedure:

1. Perform a step response experiment.
2. Approximate your step response with the process reaction curve, but only use $\tau_{63\%}$.
3. Determine A and L.
4. Calculate $\lambda = p \max(\tau, L)$, with $p = 1 \ldots 3$ Snabbhet!
5. Determine the controller parameter as

$$k_P = \frac{\tau}{A(\lambda + L)}, \qquad T_I = \tau$$

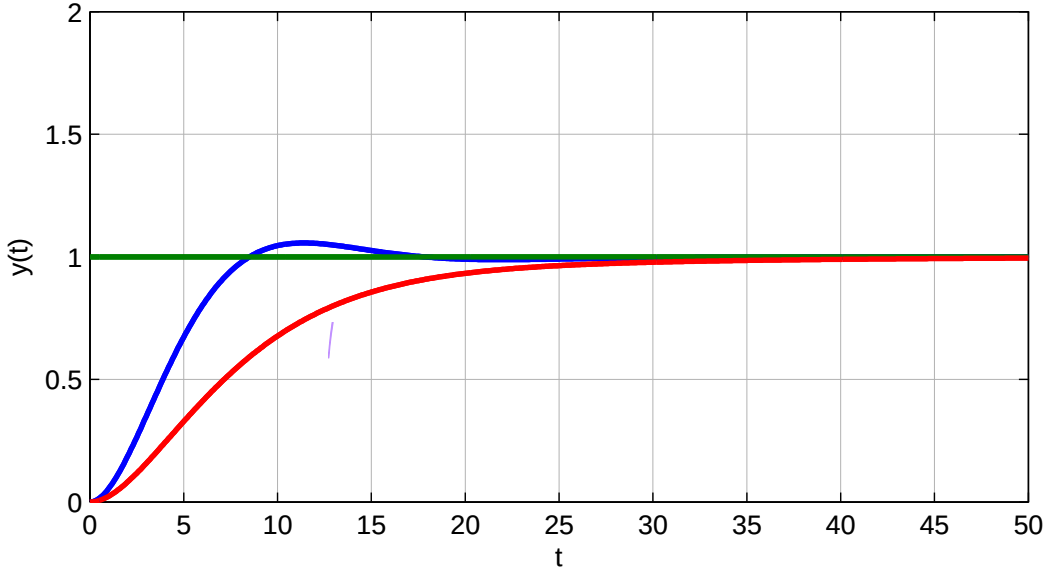
Note: This tuning should provide good closed loop performance. λ is a bandwidth parameter, which means that small values yield a faster response. Although, the guideline suggests a value of 1 to 3 for p other value can be tried. The tuning requires only the tuning of one parameter.

Analysis of the example:

Remember, the parameters of the approximation were determined as $A = 4$, $L = 1.3$ and $\tau_{63\%} = 12.9s$. From that the resulting PI controller for the two cases $p_1 = 0.2$ and $p_2 = 0.6$ are

$$D_{PI1}(s) = \frac{0.83s + 0.06}{s}, \qquad D_{PI2}(s) = \frac{0.35s + 0.02}{s}$$

The resulting response of the closed loop system for D_{PI1} (blue) and D_{PI2} (red)



Conclusion: This tuning yields a calm controller with little overshoot. If $p = 1$, then the time constant of the closed loop is similar to that of the open loop process.

λ-Method (3)

Further reading:

- Bialkowski, W.L; Weldon, AD., (October 1994): Tumregel om filter (1,3 x Ts) Digital Future of Process Control; Possibilities, Limitations, and Ramifications, Tappi Journal 77, nr 10: 69-75.
- Frederick Thomasson, Bill Bialkowski (1995): Process Control Fundamentals, chapter 6 and 7, Tappi Press.
- Tore Hägglund (1990-1997): Practical Process Control, Student literature.
- Karl Johan Åström, Tore Hägglund (1996): PID Controllers: Theory, Design, and Tuning, ISA.

Implementation of a controller

Now that we have a completely designed controller we need to implement it. There are usually, several ways to achieve this and they depend largely on the environment where the controller will operate:

- Analog implementation: Principally, not done any more!
- DCS in a process industry plant
- Embedded system or computer

DCS:

- There are standard blocks for PID Controllers
- You just have to know the parameters and the sample time at which the loop should operate.
- The standard blocks vary in their implementation between suppliers. You might need to recalculate your parameters.

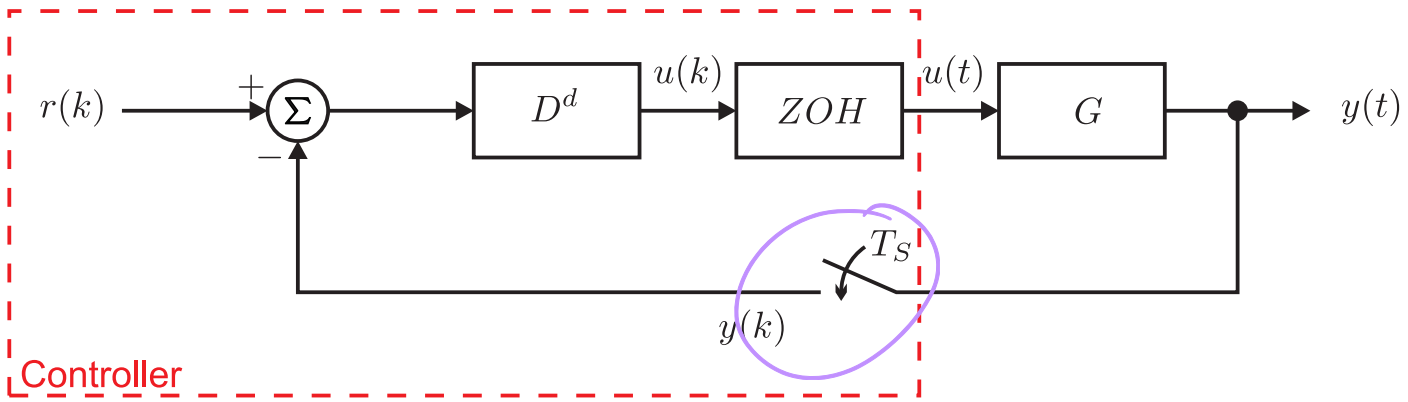
Embedded system or computer:

- There are no standard block. You need to implement the controller in a programming language
- You need to find a discrete time version of your controller.
- You need to select a sampling time.
- You need to be aware of the data storing routines of the system.

Implementation of a controller (2)

Starting point: Our controller is in continuous time and described in the laplace domain as a transfer function $D(s)$.

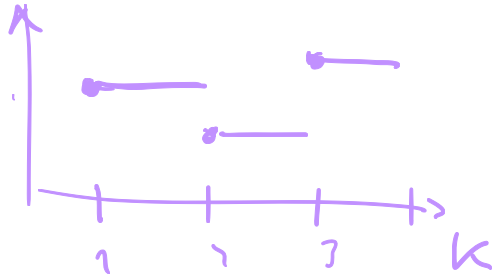
Goal: Complete control system with continuous real life process and a computer implemented controller like the following



Here we have several differences:

- The computer is running in cycles which are the instances k. Each k corresponds to a real-time t.
- Zero order hold block, which converts $u(k)$ into $u(t)$
- Sampler block, which converts $y(t)$ into $y(k)$

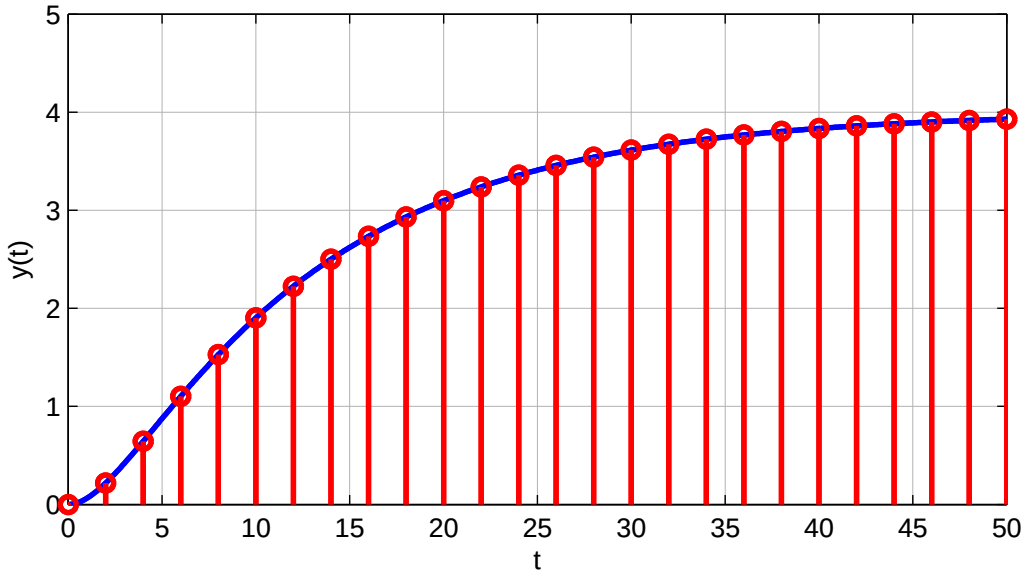
Note: We assume that the computer runs at a constant cycle time T_S



Discrete time vs continuous time

The conversion from continuous time to discrete time is done by a sampler block. In our case, $y(t)$ is converted into $y(k)$.

Here, $y(t)$ (blue) is sampled into the discrete time signal $y(k)$:



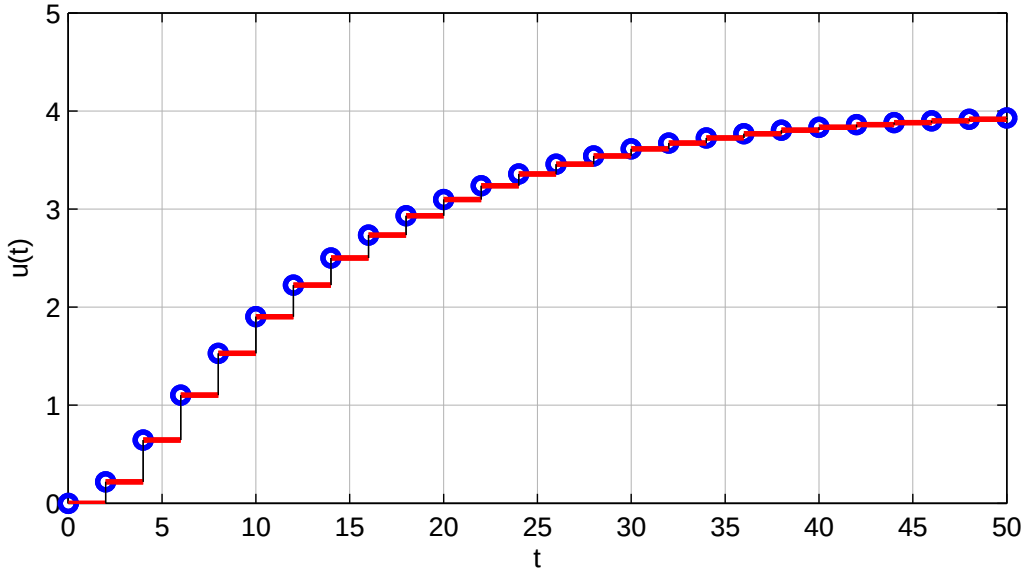
Note:

- Sampling at time t can be expressed in terms of instances k: $t = k \cdot T_S$
- Inbetween the sampling instances there is no information available to the computer program.

Discrete time vs continuous time (2)

When the computer program has determined a new control output, then this output is set for the process. Until the program returns to the same point again, there is no change in the control output.

Usually a D/A card can hold the signal that is send to the output with no further interaction with the computer. In the following figure, the output $u(k)$ (blue), output of the zero order hold in red.



Note:

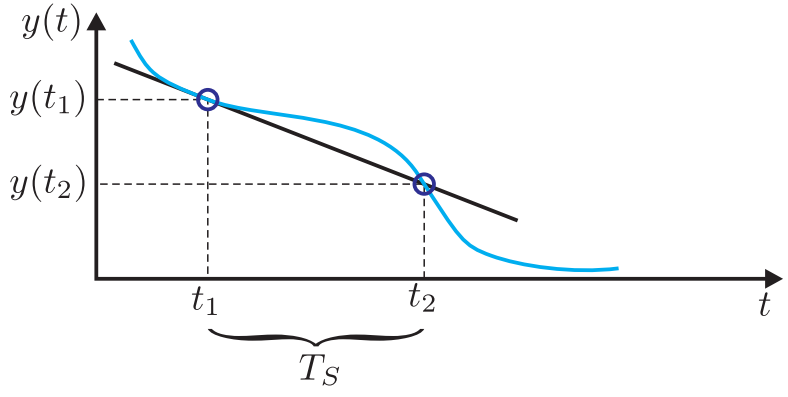
- The continuous signal $u(t)$ is fixed inbetween the execution cycles.

Discrete equivalent

In this course, we will represent the continuous time controller by a discrete time equivalent. The component in a controller that are affected by the conversion into discrete time are integral and derivative action.

Idea: We approximate the integration and derivation of the controller by a discrete time operation.

The derivative of a signal could by derived as follows:



which leads to the following

$$\dot{y}(t_1) \approx \frac{y(t_2) - y(t_1)}{T_S} = \frac{y((k+1)T_S) - y(kT_S)}{T_S}$$

Obviously, the quality of the derivative depends largely on the chosen sample time T_S. If the sample time is chosen small enough, then the approximation will not introduce large errors.

We can now introduce the shift operator z:

$$y(k+1) = zy(k)$$

$$D(s) = K_p + \frac{K_I}{s} + K_D \cdot s$$

Discrete equivalent (2)

Now we can rewrite the approximation into

$$\dot{y}(t_1) \approx \frac{zy((k)T_S) - y(kT_S)}{T_S} = y(kT_S)\frac{z-1}{T_S}$$

Remember, in the Laplace domain the derivative was the multiplication with the operator s. Consequently, we approximate the counterpart of s by $\frac{z-1}{T_S}$.

If we now replace in a transfer function the operator s by $\frac{z-1}{T_S}$, then we have found the discrete time transfer function as the so called Euler forward approximation.

There are several approximations that can be used:

1. Euler forward: $s \triangleq \frac{z-1}{T_S}$
2. Euler backward: $s \triangleq \frac{1-z^{-1}}{T_S}$
3. Tustin: $s \triangleq \frac{2(1-z^{-1})}{T_S(1+z^{-1})}$

These approximations of the derivative also correspond to the similar approximation for the integration, besides Tustin that corresponds to the trapezoid method.

Discretization of a PID controller

Euler backward is the most simplistic way to discretize and also implement the resulting controller. But in order to achieve good results, it is necessary to choose a rather high sampling rate (small sampling time T_S).

We assume that the PID controller is given in the parallel structure:

$$D(s) = k_P + \frac{k_I}{s} + k_D s$$

Now we can replace s by $\frac{1-z^{-1}}{T_S}$ and we get:

$$D^d(z) = k_P + \frac{k_I T_S}{1-z^{-1}} + \frac{k_D(1-z^{-1})}{T_S}$$

This is now the discrete time transfer function for

$$D^d(z) = \frac{u(k)}{e(k)}$$

Thus our control signal can now be calculated from the following difference equation:

$$u(k) = \left\{ k_P + \frac{k_I T_S}{1-z^{-1}} + \frac{k_D(1-z^{-1})}{T_S} \right\} e(k)$$

Here we can obviously do the calculation for each addend separately

$$\begin{cases} u_P(k) = k_P e(k) \\ u_I(k) = \frac{k_I T_S}{1-z^{-1}} e(k) \\ u_D(k) = \frac{k_D(1-z^{-1})}{T_S} e(k) \end{cases}$$

Discretization of a PID controller (2)

For the integral action $u_I(k)$ we get

$$u_I(k)(1 - z^{-1}) = k_I T_S e(k)$$
$$\Rightarrow \quad u_I(k) = u_I(k-1) + k_I T_S e(k)$$

$\Rightarrow \quad u(k) - u(k) \cdot z^{-1}$ ↳ $u(k-1)$

For the derivative action $u_D(k)$ we get

$$u_D(k) = \frac{k_D}{T_S}(1 - z^{-1})e(k)$$
$$\Rightarrow \quad u_D(k) = \frac{k_D}{T_S}(e(k) - e(k-1))$$

The complete control action can now be calculated from

$$u(k) = u_P(k) + u_I(k) + u_D(k)$$

Note:

- We need to store $u(k)$ and $e(k)$ in order to be available for the next execution of the controller code.
- The calculation of $u_I(k)$ is a simple implementation of an integrator.

Sample time selection

The last element that is missing in the controller implementation is the sample time T_S.

In this course we will only give guidelines how to select a sampling time, the follow-up course will give more details on the topic.

Guideline:

- First order process: $T_S < \dfrac{\tau}{10}$
- Second order process: $T_S < \dfrac{1}{20\omega_n}$
- Approximation with good confidence: $T_S < \dfrac{1}{30\omega_n}$

Note: If the sample time is choosen too large, then the closed loop system can become unstable.

Improvement: In order to get a better approximation, the ZOH block can be approximated by

$$G_{ZOH}(s) = \frac{1}{\frac{T_S}{2}s + 1}$$

and used in the discretization of the controller.

Actuator saturation

Any real process has limitations and especially actuators have bounds.

For example: A valve can only be fully opened or closed. A pump can either stopped or run at full speed. A radiator does not have a negative power output.

Effect on a closed loop system:

When the actuator saturates, then the control signal that is computed by the controller is NOT entering the process.

$\Downarrow$

The feedback signal is not a result of the computed control signal

$\Downarrow$

The error signal $r(t) - y(t)$ is interpreted wrong

$\Downarrow$

The integrator in the controller assumes that the error is a result of the prior control signal, which is wrong.

$\Downarrow$

A wrong control signal is computer for future control actions

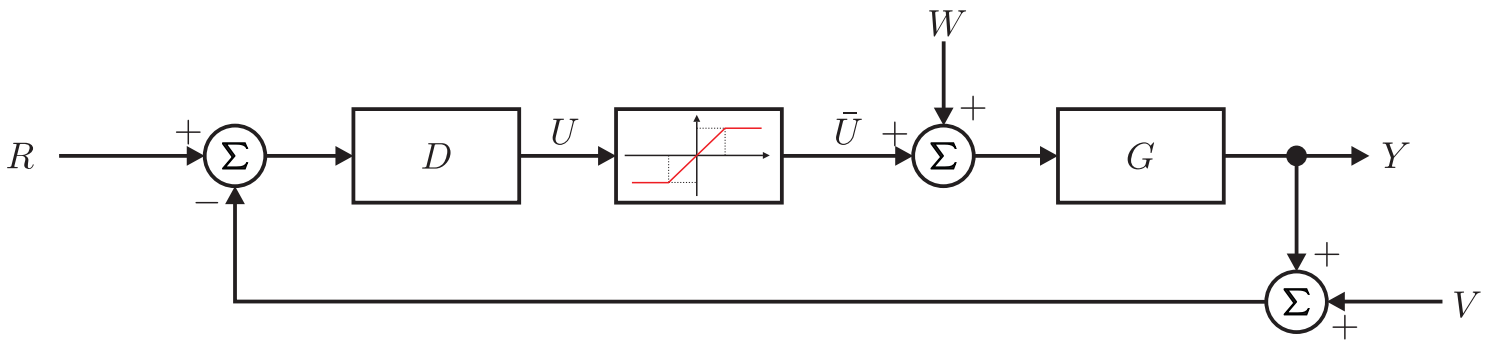
$\Downarrow$

The closed loop system does NOT behave as expected.

$\rightrightarrows$ VERY BAD!!

Actuator saturation (2)

The closed loop system with a saturation can be depicted as the following



The saturation can be represented as the following

$$\bar{u} = \begin{cases} L_u, & u \geq L_u \\ L_l, & u \leq L_l \\ u, & \text{otherwise} \end{cases}$$

If $u > L_u$, then the control error is computed as

$$E(s) = R(s) - G(s)L_u$$

otherwise if $u > L_l$ then it is computed as

$$E(s) = R(s) - G(s)U(s)$$

Thus, when the control signal is saturated, then the control error does no longer depend on the control signal any more. This is similar to an open-loop system.

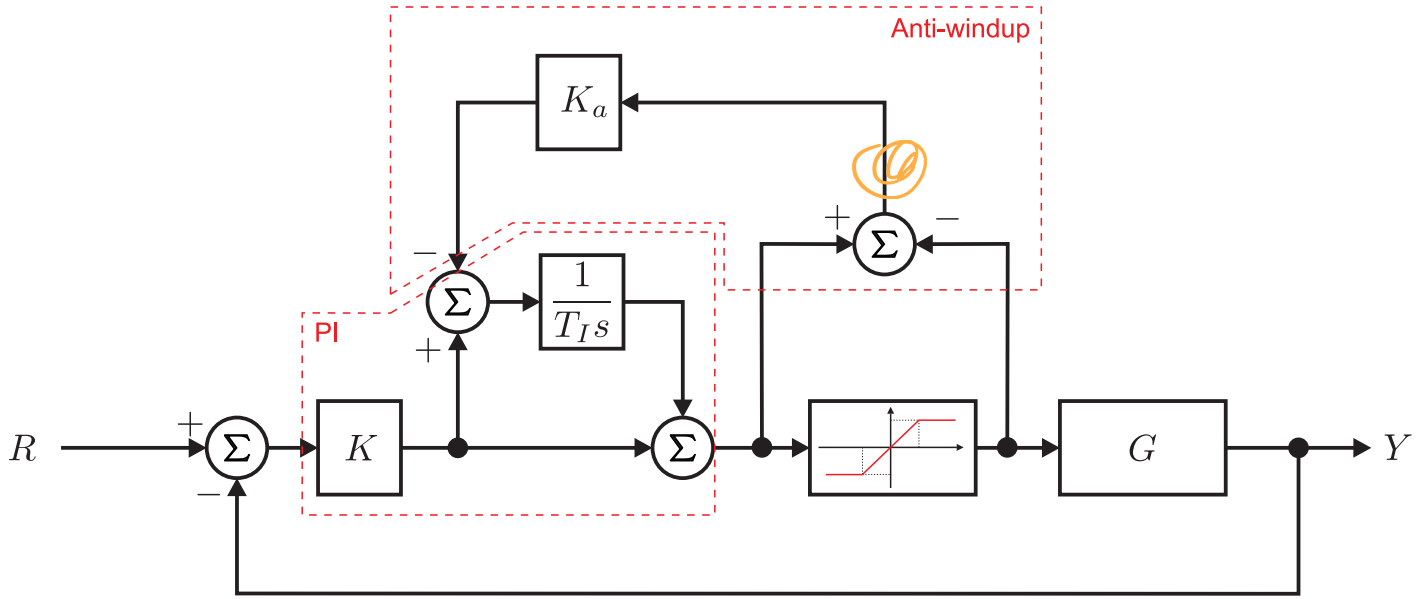
As a result, the integrators in the controller do not integrate correctly and will wind up.

Actuator saturation (3)

This problem can be dealt with in different ways. Here are two common solutions:

1. Stop the integrator as soon as the actuator is saturated.
   In this case your controller becomes a P or PD controller.

2. You could implement an anti-windup strategy, that adjusts the integrator when saturations occur.



Actuator saturation (4)

Notes:

- The gain K_a could be chosen to have a value of 1, but usually somewhat less than one is a good design choice.
- K_a is used, since the control signal u does not only depend on the integrator part and therefore is is advisable to reduce the compensation.

Let's have a look at an example:

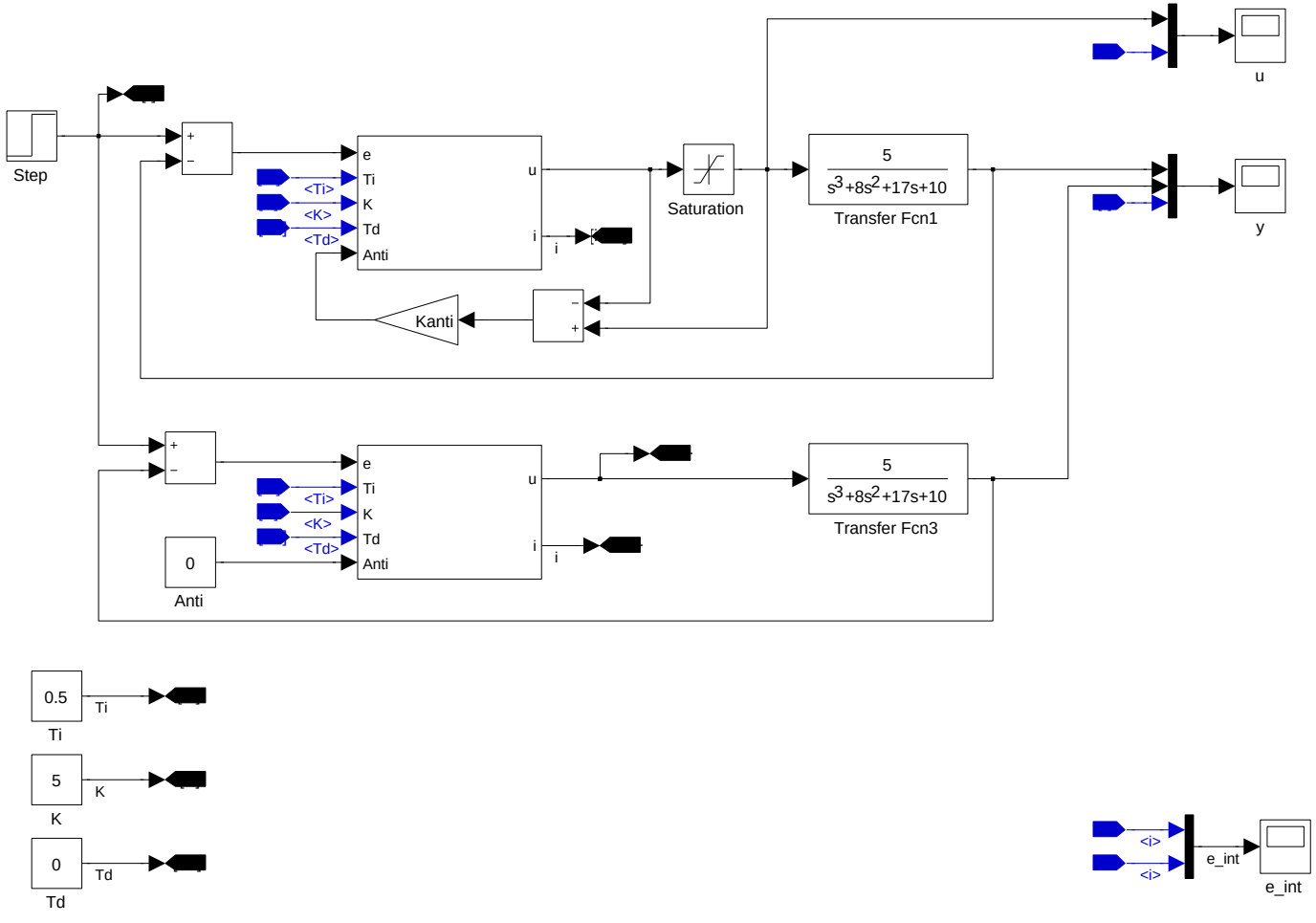
$$G(s) = \frac{5}{s^3 + 8s^2 + 17s + 10}, \qquad D(s) = 5(1 + \frac{1}{0.5s})$$

We assume that the control saturates in the following way:

$$\bar{u} = \begin{cases} 2.2, & u \geq 2.2 \\ -2.2, & u \leq -2.2 \\ u, & \text{otherwise} \end{cases}$$

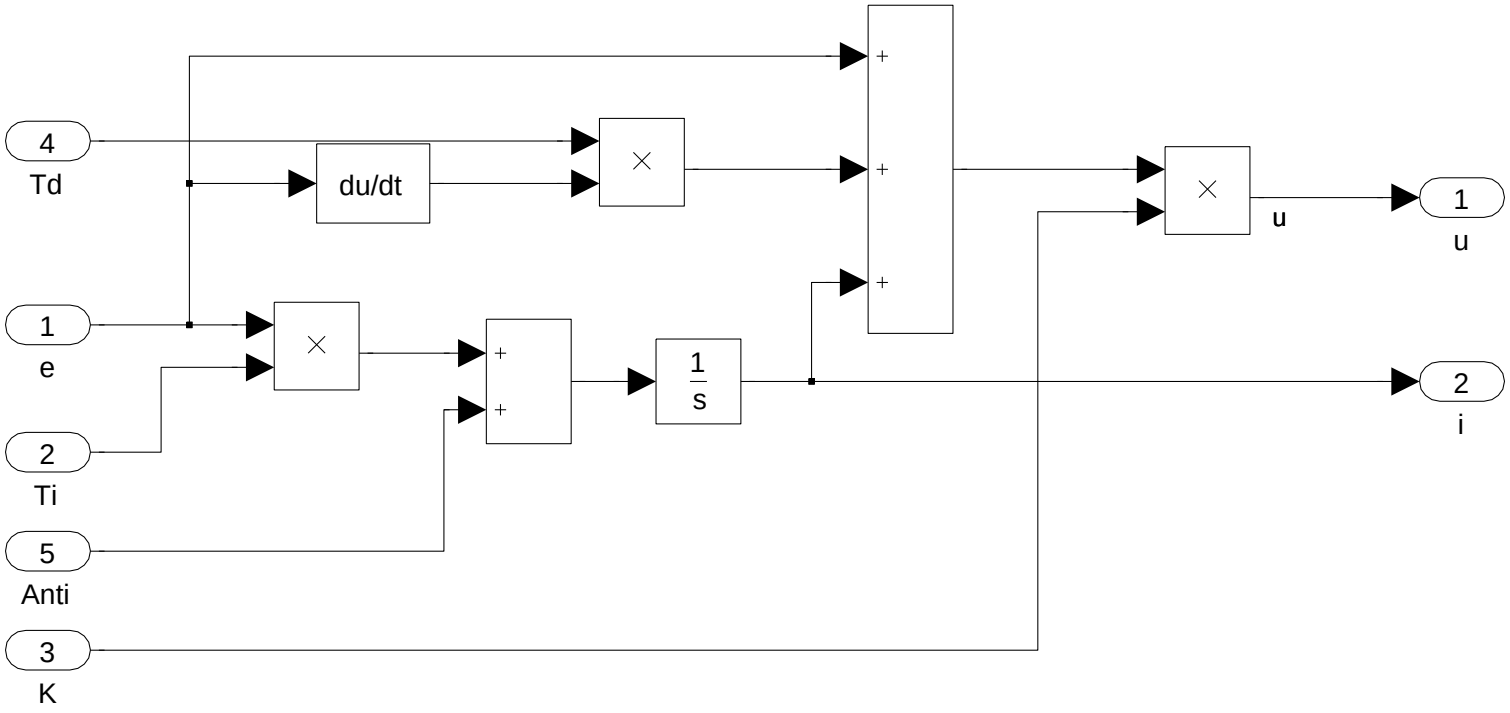
Antiwindup of a PI controller

We create two parallel closed loop control systems for comparison:



Antiwindup of a PI controller (2)

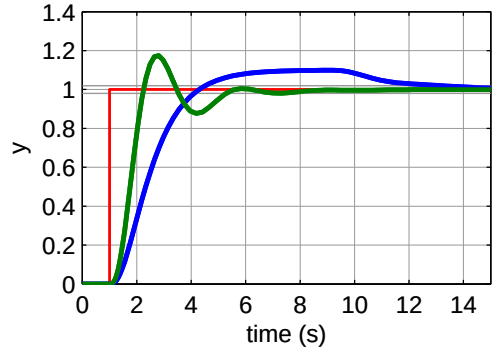
A closer look into the PI controller:

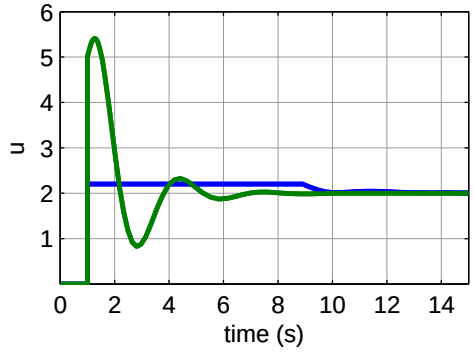


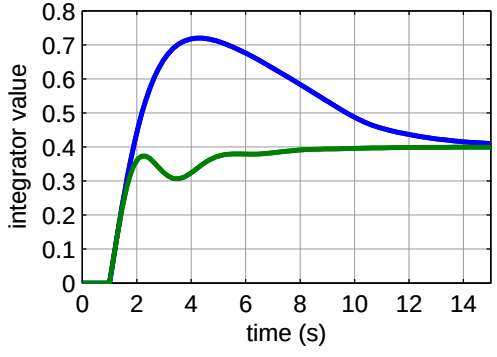
Antiwindup of a PI controller (3)

Simulation of the closed loop system without anti-windup for a unit step in the reference.

Blue curves represent the loop with saturation, green without saturation.







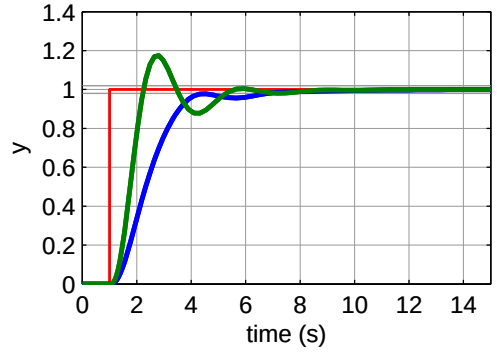
Notes:

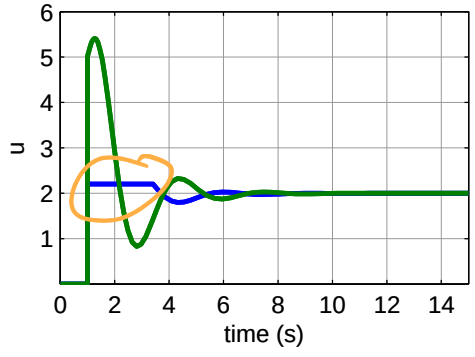
- The saturated loop converges much later and the control remains a long time in saturation.
- The integrator winds up largely.

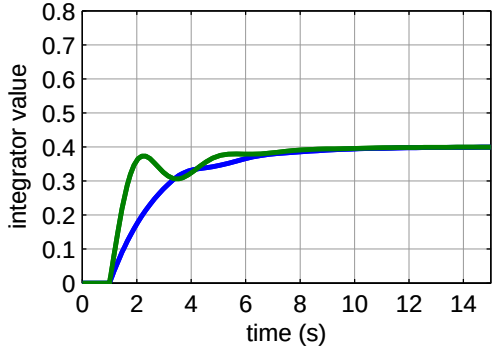
Antiwindup of a PI controller (4)

Simulation of the closed loop system with anti-windup for a unit step in the reference.

Blue curves represent the loop with saturation, green without saturation.







Notes:

- The saturated loop converges later but the control signal is only a short time saturated.
- The integrator wind up is mitigated.